

Q1. What are the four main principles of Object-Oriented Programming? Explain each with an example.

Ans. The four main principal of Object-Oriented Programming are-

1. Abstraction
2. Encapsulation
3. Inheritance
4. Polymorphism

The First Principal of Object-Oriented programming is **Abstraction.**

1. **Abstraction** is a process of hiding implementation details and exposing only the functionality to the user. In abstraction, we deal with ideas and not events. This means the user will only know “what it does” rather than “how it does”.

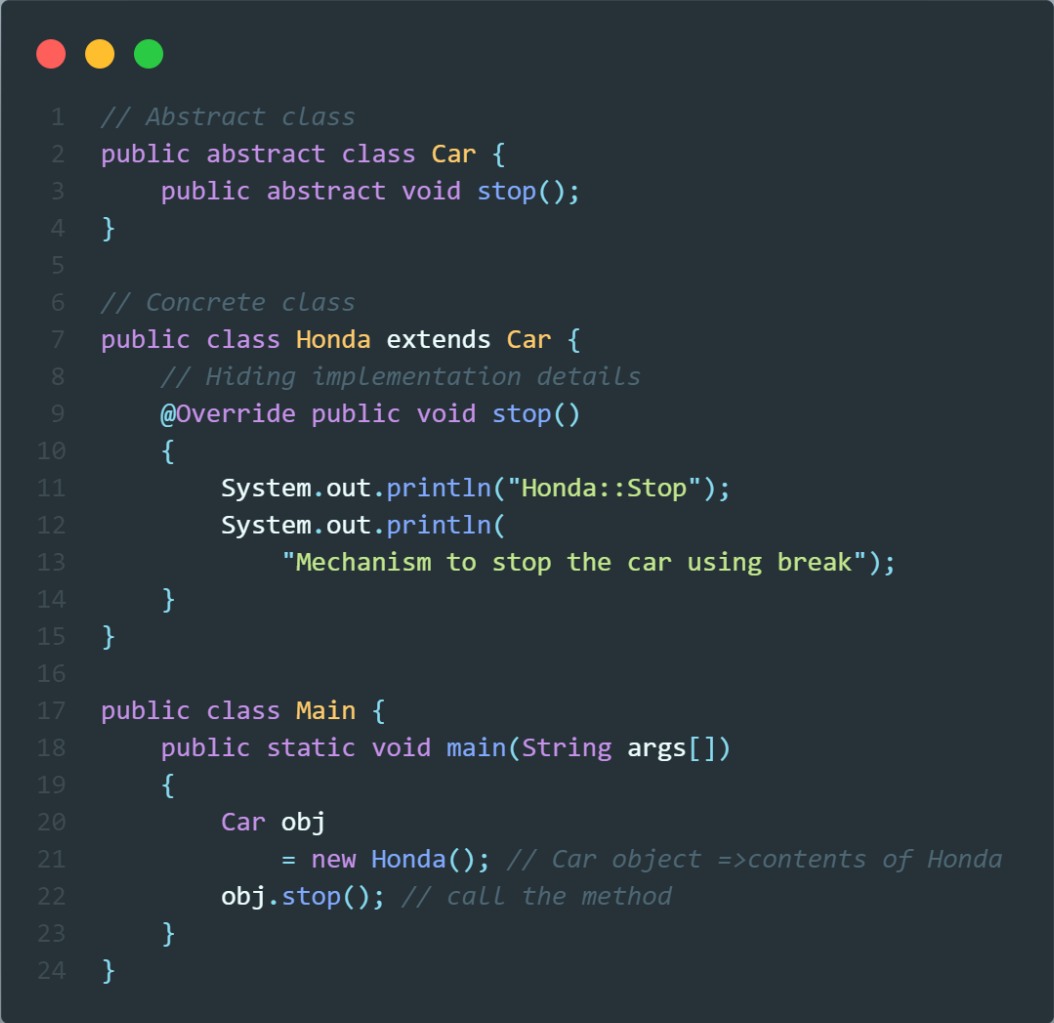
There are two ways to achieve abstraction in Java:

- Abstract class
- Interface

Abstract Class - An abstract class in Java is a class that cannot be instantiated on its own and is meant to be extended by other classes. It serves as a blueprint for other classes, allowing you to define methods that must

be implemented by subclasses, as well as methods that can have their own implementation.

For Example :



```
1  // Abstract class
2  public abstract class Car {
3      public abstract void stop();
4  }
5
6  // Concrete class
7  public class Honda extends Car {
8      // Hiding implementation details
9      @Override public void stop()
10     {
11         System.out.println("Honda::Stop");
12         System.out.println(
13             "Mechanism to stop the car using break");
14     }
15 }
16
17 public class Main {
18     public static void main(String args[])
19     {
20         Car obj
21         = new Honda(); // Car object => contents of Honda
22         obj.stop(); // call the method
23     }
24 }
```

Interfaces – An interface in Java is a blueprint for a class. It is a reference type, similar to a class, that can contain abstract methods (methods without a body) and constants (static and final variables). Interfaces are used to achieve abstraction and multiple inheritance in Java.

For Example :

```
1  // Abstract class
2  public abstract class Car {
3      public abstract void stop();
4  }
5
6  // Concrete class
7  public class Honda extends Car {
8      // Hiding implementation details
9      @Override public void stop()
10     {
11         System.out.println("Honda::Stop");
12         System.out.println(
13             "Mechanism to stop the car using break");
14     }
15 }
16
17 public class Main {
18     public static void main(String args[])
19     {
20         Car obj
21             = new Honda(); // Car object => contents of Honda
22         obj.stop(); // call the method
23     }
24 }
```

2. Encapsulation

Encapsulation is the process of wrapping code and data together into a single unit.

In order to achieve encapsulation in java follow certain steps as proposed below:

- Declare the variables as private

- Declare the setters and getters to set and get the variable values

For Example :

```
1  // A Java class which is a fully encapsulated class.
2  public class Car {
3      // private variable
4      private String name;
5
6      // getter method for name
7      public String getName(){
8          return name;
9      }
10
11     // setter method for name
12     public void setName(String name){
13         this.name = name;
14     }
15 }
16
17 public class Test {
18     public static void main(String[] args){
19
20         Car car = new Car();
21
22         car.setName("Honda");
23
24         System.out.println(car.getName());
25
26     }
27 }
```

3. Inheritance

Inheritance is the process of one class inheriting properties and methods from another class in Java. Inheritance is used when we have is-a relationship between objects. Inheritance in Java is implemented using extends keyword.

There are 5 different types of inheritance in java as follows:

- Single Inheritance: Class B inherits Class A using extends keyword
- Multilevel Inheritance: Class C inherits class B and B inherits class A using extends keyword
- Hierarchy Inheritance: Class B and C inherits class A in hierarchy order using extends keyword
- Multiple Inheritance: Class C inherits Class A and B. Here A and B both are superclass and C is only one child class. Java is not supporting Multiple Inheritance, but we can implement using Interfaces.
- Hybrid Inheritance: Class D inherits class B and class C. Class B and C inherits A. Here same again Class D inherits two superclass, so Java is not supporting Hybrid Inheritance as well.

For Example :



```
1  // super class
2  class Car {
3      public String wheelStatus;
4      public int noOfWheels;
5
6      public Car(String wheelStatus, int noOfWheels)
7      {
8          this.wheelStatus = wheelStatus;
9          this.noOfWheels = noOfWheels;
10     }
11
12     public void applyBrake()
13     {
14         wheelStatus = "Stop";
15         System.out.println("Stop the car using break");
16     }
17
18     public String toString()
19     {
20         return ("No of wheels in car " + noOfWheels + "\n"
21             + "status of the wheels " + wheelStatus);
22     }
23 }
24
25 // sub class
26 class Honda extends Car {
27     public Boolean alloyWheel;
28
29     public Honda(String wheelStatus, int noOfWheels,
30         Boolean alloyWheel)
31     {
32         super(wheelStatus, noOfWheels);
33         this.alloyWheel = alloyWheel;
34     }
35
36     public void setAlloyWheel(Boolean alloyWheel)
37     {
38         this.alloyWheel = alloyWheel;
39     }
40
41     @Override public String toString()
42     {
43         return (super.toString() + "\nCar alloy wheel "
44             + alloyWheel);
45     }
46 }
47
48 public class Main {
49     public static void main(String args[])
50     {
51
52         Honda honda = new Honda("stop", 4, true);
53         System.out.println(honda.toString());
54     }
55 }
```

4. Polymorphism

Polymorphism is the ability to perform many things in many ways. The word Polymorphism is from two different Greek words- poly and morphs. “Poly” means many, and “Morphs” means forms. So polymorphism means many forms. The polymorphism can be present in the case of inheritance also. The functions behave differently based on the actual implementation.

There are two types of polymorphism as listed below:

1. Static or Compile-time Polymorphism
2. Dynamic or Run-time Polymorphism

Static or Compile-time Polymorphism

Static or Compile-time Polymorphism when the compiler is able to determine the actual function, it's called compile-time polymorphism. Compile-time polymorphism can be achieved by method overloading in java.

3. For Example :



```
1 public class Car{
2     public void speed() {}
3     public void speed(String accelerator) {}
4     public int speed(String accelerator, int speedUp) {
5         return carSpeed;
6     }
7 }
```

Dynamic or Run-time Polymorphism

Dynamic (or run-time) polymorphism occurs when the compiler is not able to determine at compile-time which method (superclass or subclass) will be called. This decision is made at run-time. Run-time polymorphism is achieved through method overriding, which happens when a method in a subclass has the same name, return type, and parameters as a method in its superclass.

When the superclass method is overridden in the subclass, it is called method overriding.

For Example :


```
1  import java.util.Random;
2
3  class DeliveryBoy {
4
5      public void deliver() {
6          System.out.println("Delivering Item");
7      }
8
9      public static void main(String[] args) {
10         DeliveryBoy deliveryBoy = getDeliveryBoy();
11         deliveryBoy.deliver();
12     }
13
14     private static DeliveryBoy getDeliveryBoy() {
15         Random random = new Random();
16         int number = random.nextInt(5);
17         return number % 2 == 0 ? new Postman() : new FoodDeliveryBoy();
18     }
19 }
20
21 class Postman extends DeliveryBoy {
22     @Override
23     public void deliver() {
24         System.out.println("Delivering Letters");
25     }
26 }
27
28 class FoodDeliveryBoy extends DeliveryBoy {
29     @Override
30     public void deliver() {
31         System.out.println("Delivering Food");
32     }
33 }
```

Q2. How does inheritance work in OOP?
Write a sample program to demonstrate single and multilevel inheritance.

Inheritance is the process of one class inheriting properties and methods from another class in Java. Inheritance is used when we have is-a relationship between objects. Inheritance in Java is implemented using extends keyword.

There are 5 different types of inheritance in java as follows:

- Single Inheritance: Class B inherits Class A using extends keyword
- Multilevel Inheritance: Class C inherits class B and B inherits class A using extends keyword
- Hierarchy Inheritance: Class B and C inherits class A in hierarchy order using extends keyword
- Multiple Inheritance: Class C inherits Class A and B. Here A and B both are superclass and C is only one child class. Java is not supporting Multiple Inheritance, but we can implement using Interfaces.
- Hybrid Inheritance: Class D inherits class B and class C. Class B and C inherits A. Here same again Class D inherits two superclass, so Java is not supporting Hybrid Inheritance as well.

For Example :

```
1  // super class
2  class Car {
3      public String wheelStatus;
4      public int noOfWheels;
5
6      public Car(String wheelStatus, int noOfWheels)
7      {
8          this.wheelStatus = wheelStatus;
9          this.noOfWheels = noOfWheels;
10     }
11
12     public void applyBrake()
13     {
14         wheelStatus = "Stop";
15         System.out.println("Stop the car using break");
16     }
17
18     public String toString()
19     {
20         return ("No of wheels in car " + noOfWheels + "\n"
21             + "status of the wheels " + wheelStatus);
22     }
23 }
24
25 // sub class
26 class Honda extends Car {
27     public Boolean alloyWheel;
28
29     public Honda(String wheelStatus, int noOfWheels,
30         Boolean alloyWheel)
31     {
32         super(wheelStatus, noOfWheels);
33         this.alloyWheel = alloyWheel;
34     }
35
36     public void setAlloyWheel(Boolean alloyWheel)
37     {
38         this.alloyWheel = alloyWheel;
39     }
40
41     @Override public String toString()
42     {
43         return (super.toString() + "\nCar alloy wheel "
44             + alloyWheel);
45     }
46 }
47
48 public class Main {
49     public static void main(String args[])
50     {
51
52         Honda honda = new Honda("stop", 4, true);
53         System.out.println(honda.toString());
54     }
55 }
```

Q3. What is the difference between method overloading and method overriding? Provide code examples.

Ans. Method Overloading

- Occurs within the same class.
- Involves defining multiple methods with the same name but different parameters (either in number, type, or order).
- Implemented at compile-time (static binding).
- Helps in improving code readability and flexibility.

Example of Method Overloading in Java



```
1  class MathOperations {
2      // Method with one parameter
3      int multiply(int a) {
4          return a * a;
5      }
6
7      int multiply(int a, int b) {
8          return a * b;
9      }
10 }
11 public class TestOverloading {
12     public static void main(String[] args) {
13         MathOperations obj = new MathOperations();
14         System.out.println(obj.multiply(5));
15         System.out.println(obj.multiply(3, 4));
16     }
17 }
18
```

Method Overriding

- Occurs **between parent and child classes**.
- Allows a subclass to **provide a specific implementation** of a method defined in its superclass.
- Implemented at **runtime** (dynamic binding).
- Supports **polymorphism**, enabling behavior customization in child classes.

Example of Method Overriding in Java

```
1  class Animal {
2      void makeSound() {
3          System.out.println("Animal makes a sound");
4      }
5  }
6
7  class Dog extends Animal {
8      @Override
9      void makeSound() { // Overriding the parent method
10         System.out.println("Dog barks");
11     }
12 }
13
14 public class TestOverriding {
15     public static void main(String[] args) {
16         Animal obj1 = new Animal();
17         obj1.makeSound(); // Calls parent class method
18
19         Animal obj2 = new Dog();
20         obj2.makeSound(); // Calls overridden method in child class
21     }
22 }
23
```

Q4. What is encapsulation and how does it help in software development? Show how it is implemented in code.

Ans. Encapsulation is a fundamental concept in object-oriented programming (OOP) that helps bundle data (variables) and methods that operate on the data into a single unit—typically a class. It restricts direct access to an object's data and allows controlled modification through methods, ensuring better security and maintainability.

Benefits of Encapsulation

1. **Data Security** – Prevents unauthorized access by restricting visibility.
2. **Code Maintainability** – Encapsulated code is easier to modify without affecting other parts of the program.
3. **Modular Approach** – Promotes a well-structured design with clear responsibilities.
4. **Reusability** – Encourages using encapsulated components across different programs.

Example of Encapsulation in Java

Encapsulation is implemented using private variables and getter/setter methods for controlled access.



```
1  class BankAccount {
2      private double balance; // Private variable ensures restricted access
3
4      // Constructor
5      public BankAccount(double initialBalance) {
6          this.balance = initialBalance;
7      }
8
9      // Getter method for balance
10     public double getBalance() {
11         return balance;
12     }
13
14     // Setter method to update balance safely
15     public void deposit(double amount) {
16         if (amount > 0) {
17             balance += amount;
18             System.out.println("Deposited: " + amount);
19         } else {
20             System.out.println("Invalid deposit amount!");
21         }
22     }
23 }
24
25 public class TestEncapsulation {
26     public static void main(String[] args) {
27         BankAccount account = new BankAccount(5000);
28
29         // Accessing balance using getter
30         System.out.println("Balance: " + account.getBalance());
31
32         // Depositing money using setter
33         account.deposit(1500);
34         System.out.println("Updated Balance: " + account.getBalance());
35     }
36 }
37
```